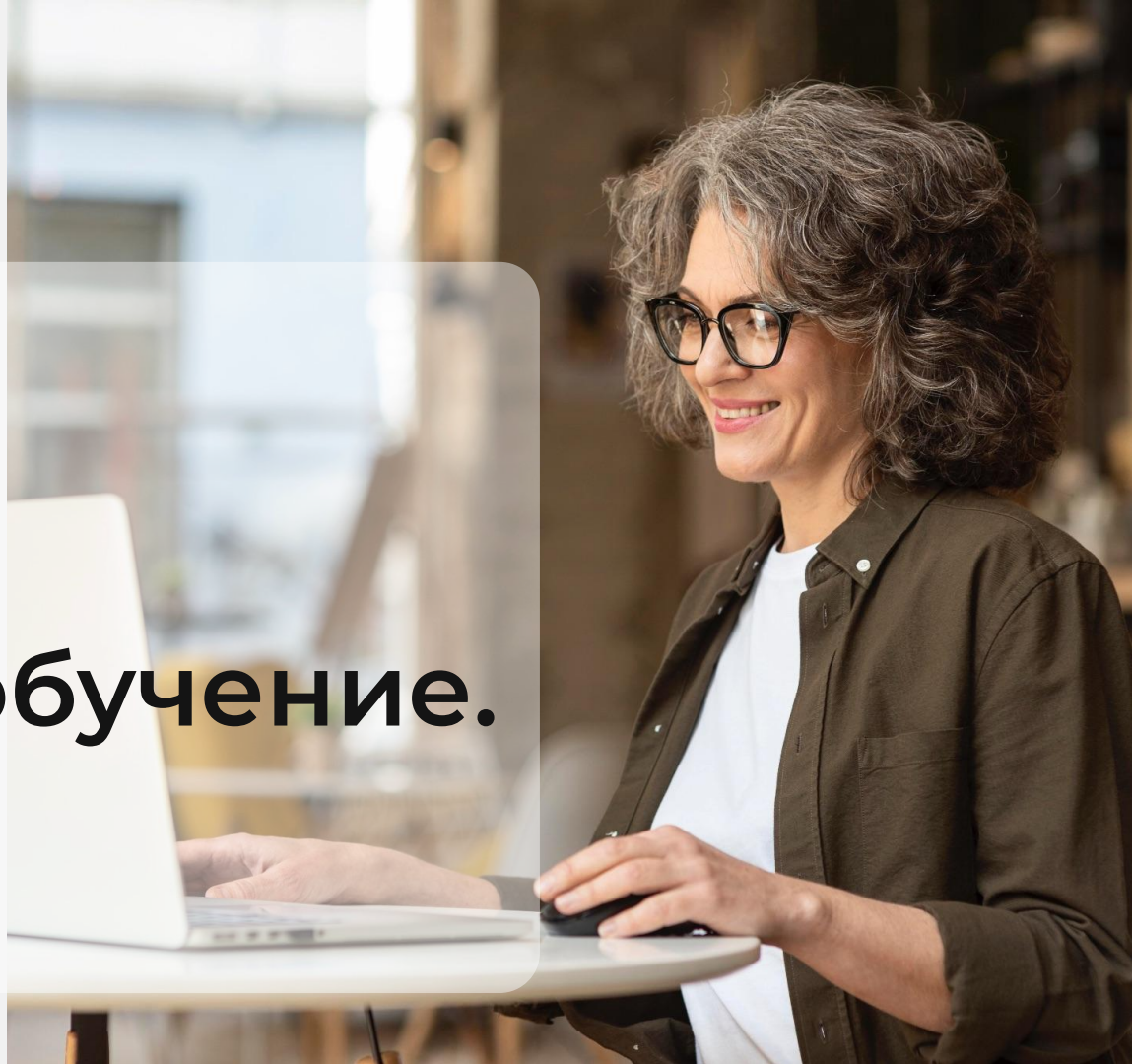


Python
for DA

Введение в машинное обучение. Обучение с учителем



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение



Машинное обучение: определение и основные понятия



Типы машинного обучения: обучение с учителем, обучение без учителя и обучение с подкреплением



Разделение данных на обучающие и тестовые наборы



Реализация с помощью функции `train_test_split` из Scikit-Learn

План занятия

- Линейная регрессия: Понятие и математические основы
- Линейная регрессия: Реализация в Python
- Логистическая регрессия: Понятие и математические основы
- Логистическая регрессия: Реализация на Python
- Деревья решений: Концепция и принцип работы
- Деревья решений: Реализация на Python
- Ближайшие соседи (K-Nearest Neighbors, KNN):
- Концепция и принцип работы K-Nearest Neighbors (KNN): Реализация в Python



ОСНОВНОЙ БЛОК





Линейная регрессия: Понятие и основы



Линейная регрессия

один из самых простых и широко используемых алгоритмов в машинном обучении и статистике. Это метод контролируемого обучения, используемый для прогнозирования непрерывной итоговой переменной (зависимой переменной) на основе одной или нескольких предикторных переменных (независимых переменных)

Цель линейной регрессии



Найти наилучшим образом подходящую прямую линию через точки данных, которая может быть использована для прогнозирования.

Ключевые термины

Зависимая переменная (Y)

Переменная, которую мы пытаемся предсказать.

Независимая переменная (X)

Переменная(ые), используемая(ые) для прогнозирования.

Линия регрессии

Линия наилучшего соответствия, которая минимизирует разницу между фактическими точками данных и предсказанными значениями.

Взаимосвязь между переменными

$$y=mx+b,$$

где:

y - прогнозируемое значение зависимой переменной.

x - независимая переменная.

m - наклон линии (насколько y изменяется на единицу изменения в x).

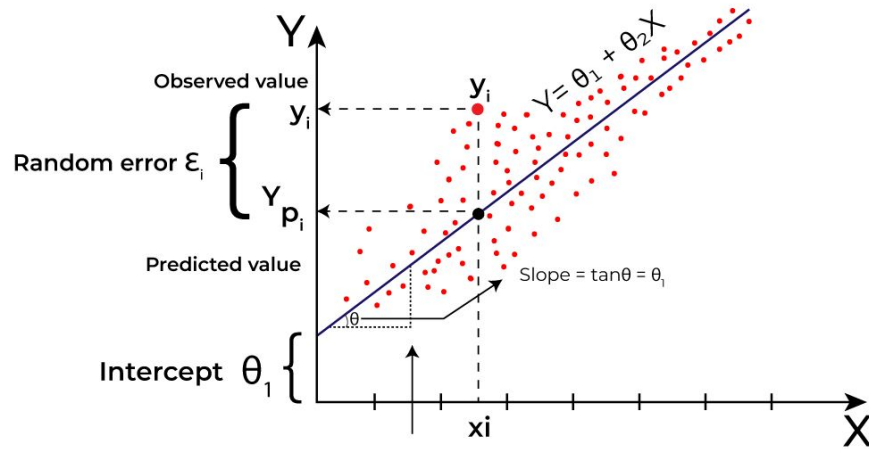
b - y -интерцепт (значение y , когда x равно 0).



Регрессия по методу наименьших квадратов регрессия

Линия наилучшего соответствия определяется путем минимизации суммы квадратов разностей между фактическими и предсказанными значениями

Регрессия по методу наименьших квадратов





Средняя квадратичная ошибка (MSE)

Измеряет среднее значение квадратов ошибок (разницы между фактическими и предсказанными значениями)

Определение MSE

$$J(m, b) = \frac{1}{N} \sum_{i=1}^N (y_i - (mx_i + b))^2,$$

где y_i - фактическое значение, $mx_i + b$ - прогнозируемое значение.

Задача состоит в том, чтобы найти значения m и b , которые минимизируют эту функцию потерь.



Градиентный спуск

это алгоритм оптимизации, используемый для минимизации функции потерь. Он итеративно регулирует параметры m и b , чтобы найти минимальное значение функции потерь. Этот алгоритм имеет параметр α , который обозначает скорость обучения



ВОПРОСЫ





Линейная регрессия: Реализация в Python

Шаг 1: Импорт необходимых библиотек

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Шаг 2: Загрузка набора данных

```
# Создание примера набора данных
data = {
    'Mother': [170.2, 168.9, 162.6, 162.6, 148.6, 172.7],
    'Daughter': [175.3, 166.4, 172.7, 170.2, 158.8, 176.5]
}
df = pd.DataFrame(data)
```

Шаг 3: Изучение данных

```
# Отообразим первые несколько строк набора данных  
print(df.head())
```

```
# Сводная статистика  
print(df.describe())
```

Шаг 4: Разбиение на наборы

```
# Определяем переменные предиктора (X) и отклика (y)
X = df[['Mother']]
y = df['Daughter']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Шаг 5: Обучение модели

```
# Инициализируем модель линейной регрессии
model = LinearRegression()

# Обучаем модель на обучающих данных
model.fit(X_train, y_train)
```

Шаг 6: Оценка модели

```
# Делаем прогнозы на тестовом наборе
y_pred = model.predict(X_test)

# Рассчитайте метрики оценки
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")
```


Шаг 7: Визуализация результатов

```
# Постройте график обучающих данных и линии регрессии
plt.scatter(X_train, y_train, color='blue', label='Обучающие
данные')
plt.scatter(X_test, y_test, color='green', label='Тестовые данные')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=2,
label='Линия регрессии')
plt.xlabel('Рост матери (см)')
plt.ylabel('Рост дочери (см)')
plt.legend()
plt.show()
```



ВОПРОСЫ





Логистическая регрессия: Понятие и основы



Логистическая регрессия

это фундаментальный алгоритм машинного обучения, используемый для решения задач бинарной классификации. В отличие от линейной регрессии, которая предсказывает непрерывные значения, логистическая регрессия предсказывает вероятность того, что заданный входной сигнал принадлежит к одному из двух возможных классов.

Ключевые термины

Бинарная классификация

Логистическая регрессия в основном используется для решения задач бинарной классификации, когда переменная результата может принимать только два возможных значения, например «да» или «нет», «спам» или «не спам», «болезнь» или «не болезнь».

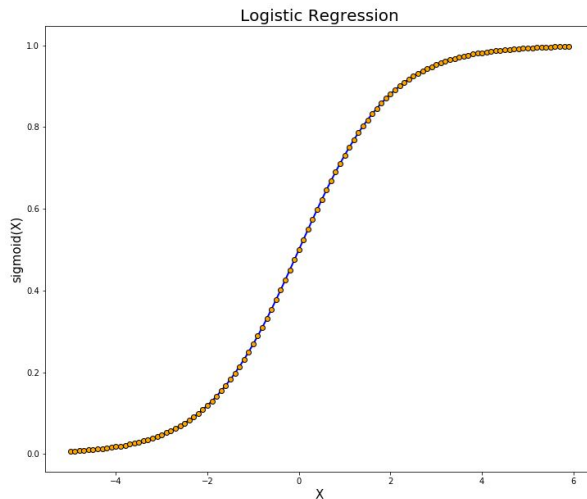
Вероятностное прогнозирование

Цель логистической регрессии - предсказать вероятность того, что данный входной сигнал принадлежит к определенному классу.

Логистическая функция

В основе логистической регрессии лежит логистическая функция, также известная как сигмоидная функция. Эта функция отображает любое вещественное число на значение между 0 и 1, что делает ее подходящей для моделирования вероятностей.

$$\sigma(z) = \frac{1}{1+e^{-z}}.$$



Линейная комбинация входов

В логистической регрессии входными данными для сигмоидальной функции z является линейная комбинация входных признаков. Это похоже на линейное уравнение, используемое в линейной регрессии:

$$z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p,$$

где x_i - входные признаки.

Модель логистической регрессии

Модель логистической регрессии объединяет линейное уравнение с сигмоидальной функцией для прогнозирования вероятности того, что данный вход принадлежит к положительному классу (например, классу 1):

$$P(y = 1|x) = \sigma(z).$$

Это уравнение дает вероятность того, что выход y равна 1 при наличии входных признаков x .

Граница принятия решения

Чтобы принять решение о бинарной классификации, мы устанавливаем порог (обычно 0,5). Если предсказанная вероятность больше или равна порогу, мы относим входные данные к положительному классу (1). В противном случае мы классифицируем его как отрицательный класс (0):

Если $P(y = 1|x) \geq 0.5$, предскажите $y=1$.

Если $P(y = 1|x) < 0.5$, предскажите $y=0$.

Функция потерь и оптимизация

Для обучения модели логистической регрессии нам необходимо найти оптимальные значения коэффициентов $(\beta_0, \beta_1, \dots, \beta_p)$. Это делается путем минимизации функции потерь, которая измеряет, насколько хорошо предсказания модели соответствуют фактическим результатам.

$$J(\beta) = - \frac{1}{N} \sum_{i=1}^N [y_i \log(P(y_i|x_i)) + (1 - y_i) \log(1 - P(y_i|x_i))].$$

Задача состоит в том, чтобы найти коэффициенты β , которые минимизируют эту функцию стоимости. Обычно это делается с помощью алгоритмов оптимизации, таких как градиентный спуск.



ВОПРОСЫ





Логистическая регрессия: Реализация на Python

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

Шаг 2: Загрузка и подготовка набора данных

```
# Создание примера набора данных
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Pass_Fail': [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
```

Шаг 3: Выполните разделение

```
# Определяем переменные-предикторы (X) и переменные-ответы (y)
X = df[['Hours_Studied']]
y = df['Pass_Fail']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Шаг 4: Обучение модели

```
# Инициализируем модель логистической регрессии  
model = LogisticRegression()  
  
# Обучаем модель на обучающих данных  
model.fit(X_train, y_train)
```


Шаг 5: Создание прогнозов

```
# Делаем предсказания на тестовом наборе  
y_pred = model.predict(X_test)
```

Шаг 6: Оценка модели

```
# Вычислить точность
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)

# Отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))
```

Шаг 7: Визуализация результатов

```
# Постройте график точек данных
plt.scatter(X, y, color='blue', label='Data points')

# Построение кривой логистической регрессии
X_range = np.linspace(X.min(), X.max(), 300)
y_prob = model.predict_proba(X_range.reshape(-1, 1))[:, 1]
plt.plot(X_range, y_prob, color='red', linewidth=2, label='Кривая
логистической регрессии')

plt.xlabel('Изученные часы')
plt.ylabel('Вероятность сдачи')
plt.legend()
plt.show()
```



ВОПРОСЫ





Деревья решений: Концепция и принцип работы



Деревья решений

это популярный и интуитивно понятный алгоритм машинного обучения, используемый для решения задач классификации и регрессии. Они называются «деревьями», поскольку используют древовидную модель решений и их возможных последствий, включая случайные исходы событий, затраты ресурсов и полезность.

Ключевые термины

Узлы

Каждый узел представляет собой признак (атрибут) в наборе данных.

Грани

Каждое ребро представляет собой правило принятия решения, основанное на признаке.

Листья

Каждый узел листа представляет результат (метку класса или непрерывное значение).

Разбиение данных



Алгоритм дерева решений работает путем рекурсивного разбиения данных на подмножества на основе значений признаков. Цель состоит в том, чтобы создать подмножества, максимально однородные по отношению к целевой переменной (переменной, которую мы хотим предсказать).

Выбор наилучшего разбиения

Индекс разнообразия Джини

Измеряет вероятность неправильной классификации случайно выбранного элемента, если он был случайно помечен в соответствии с распределением меток в подмножестве.

Энтропия

Измеряет количество неопределенности или случайности в подмножестве.

Информационный выигрыш

Измеряет уменьшение энтропии или примеси после разбиения.

Создание узлов принятия решений и узлов листьев

Узлы принятия решений

ти узлы представляют признак и условие, используемые для разделения данных. Каждый узел принятия решения имеет две или более ветвей.

Узлы листьев

Эти узлы представляют конечный результат или предсказание. В задачах классификации они представляют метки классов. В задачах регрессии они представляют непрерывные значения.

Рекурсивное разбиение



Процесс разбиения данных и создания узлов принятия решений и узлов листа повторяется рекурсивно для каждого подмножества.

Преимущества деревьев принятия решений



Легко понять и интерпретировать



Работает как с числовыми, так и с категориальными данными



Непараметрические

Недостатки деревьев принятия решений



Склонность к переоценке



Чувствительность к изменениям данных



Предвзятое отношение к признакам с большим количеством уровней



ВОПРОСЫ





Деревья решений: Реализация на Python

Шаг 1: Импорт необходимых библиотек

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```


Шаг 2: Загрузка и подготовка данных

```
# Создание примера набора данных
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Attendance': ['Плохая', 'Плохая', 'Плохая', 'Хорошая',
                  'Хорошая', 'Хорошая', 'Хорошая', 'Хорошая', 'Плохая'],
    'Pass_Fail': [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)

# Преобразуем категориальные данные в числовые
df['Attendance'] = df['Attendance'].map({'Плохая': 0, 'Хорошая':
1})# Создание примера набора данных
```

Шаг 3: Выполните разделения данных

```
# Определяем переменные-предикторы (X) и переменные-ответы (y)
X = df[['Hours_Studied', 'Attendance']]
y = df['Pass_Fail']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Шаг 4: Обучение модели

```
# Инициализируем классификатор дерева решений  
model = DecisionTreeClassifier(random_state=42)
```

```
# Обучить модель на обучающих данных  
model.fit(X_train, y_train)
```

Шаг 5: Предсказание

```
# Делаем предсказания на тестовом наборе  
y_pred = model.predict(X_test)
```

Шаг 6: Оценка модели

```
# Вычислить точность
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)

# Отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))
```

Шаг 7: Визуализация

```
# Постройте дерево решений
plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=['Hours_Studied', 'Attendance'],
class_names=['Fail', 'Pass'], filled=True)
plt.show()
```



ВОПРОСЫ





Ближайшие соседи (K-Nearest Neighbors, KNN)



К ближайших соседей (KNN)

это простой, интуитивно понятный и широко используемый алгоритм машинного обучения для задач классификации и регрессии. Это тип обучения на основе экземпляров, когда алгоритм делает прогнозы на основе сходства между входными данными и данными обучения

Ключевые термины

Обучение на основе экземпляров

KNN - это алгоритм обучения на основе экземпляров, что означает, что он запоминает обучающий набор данных и делает прогнозы на основе сохраненных экземпляров.

Ленивое обучение

KNN считается ленивым алгоритмом обучения, поскольку он не строит явную модель на этапе обучения. Вместо этого он откладывает вычисления до получения запроса.

Мера сходства

KNN использует метрику расстояния (например, евклидово расстояние) для измерения сходства между точками данных.

Выбор количества соседей (K)



Первым шагом в KNN является выбор количества соседей (K). Это гиперпараметр, определяющий, сколько ближайших соседей будет учитываться при составлении прогноза.

Влияние K на производительность алгоритма

- Малое значение K : небольшое значение K (например, $K=1$) может привести к созданию модели, чувствительной к шуму в данных, что может вызвать перебор.
- Большое значение K . Большое значение K может сгладить предсказания, но также может привести к недостаточной подгонке, когда модель не сможет уловить важные закономерности в данных.

Вычисление расстояния



$$\text{Евклидово расстояние} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2},$$

где x_i и y_i - значения признаков для точки входных данных и точки обучающих данных, соответственно, n - количество признаков.

Поиск ближайших соседей



После вычисления расстояний KNN определяет K ближайших соседей точки входных данных. Это K точек данных в обучающем наборе данных, которые имеют наименьшее расстояние до точки входных данных

Составление прогнозов

- Классификация: Для задач классификации KNN присваивает метку класса, которая является наиболее распространенной среди K ближайших соседей. Для этого используется механизм голосования по большинству голосов.
- Регрессия: Для задач регрессии KNN предсказывает значение путем усреднения значений K ближайших соседей.

Преимущества KNN



Простота и интуитивность



Отсутствие фазы обучения



Универсальность

Недостатки KNN



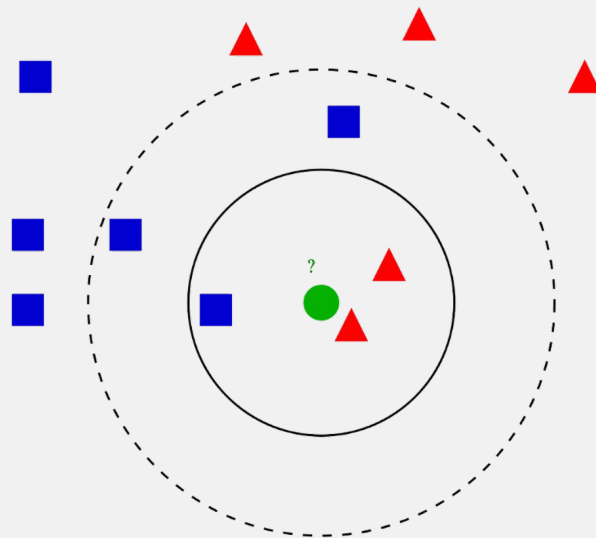
Вычислительные затраты



Чувствительность к нерелевантным признакам



Выбор





ВОПРОСЫ





K-Nearest **Neighbors** (KNN): Реализация в Python

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

Шаг 2: Загрузка и подготовка данных

```
# Создание примера набора данных
data = {
    'Weight': [150, 160, 170, 180, 190, 200, 210, 220, 230, 240],
    'Color_Score': [0.8, 0.85, 0.9, 0.95, 1.0, 0.75, 0.7, 0.65, 0.6,
0.55],
    'Label': [0, 0, 0, 0, 0, 1, 1, 1, 1, 1] # 0 для яблока, 1 для
апельсина
}
df = pd.DataFrame(data)
```

Шаг 3: Визуализация набора данных

```
# Построение графика точек данных
plt.scatter(df['Weight'], df['Color_Score'], c=df['Label'],
            cmap='coolwarm', edgecolors='k')
plt.xlabel('Вес')
plt.ylabel('Цветовой балл')
plt.title('Классификация фруктов')
plt.show()
```

Шаг 4: Выполните разбиение

```
# Определяем переменные предиктора (X) и отклика (y)
X = df[['Weight', 'Color_Score']]
y = df['Label']

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Шаг 5: Обучение KNN-модели

```
# Инициализируем KNN-классификатор
knn = KNeighborsClassifier(n_neighbors=3)

# Обучить модель на обучающих данных
knn.fit(X_train, y_train)
```


Шаг 6: Предсказание

```
# Делаем предсказания на тестовом наборе  
y_pred = knn.predict(X_test)
```

Шаг 7: Оценка модели

```
# Вычисляем точность
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)

# Отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))
```

Шаг 8: Визуализация

```
# Построение графика границы принятия решения
h = 0.01 # размер шага в сетке
x_min, x_max = X['Weight'].min() - 1, X['Weight'].max() + 1
y_min, y_max = X['Color_Score'].min() - 0.1, X['Color_Score'].max()
+ 0.1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min,
y_max, h))
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X['Weight'], X['Color_Score'], c=y, edgecolors='k',
cmap='coolwarm')
plt.xlabel('Вес')
plt.ylabel('Цветовая оценка')
plt.title('Граница принятия решения для KNN')
plt.show()
```



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание 1



Используйте набор данных Iris, чтобы предсказать вид цветков ириса на основе их физических характеристик.

Решение

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')
print('Classification Report:')
print(classification_report(y_test, y_pred))
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
```



ВОПРОСЫ



Домашнее задание

1. Используйте набор данных California Housing, чтобы сравнить эффективность линейной регрессии и деревьев решений при прогнозировании цен на жилье.
2. Используйте набор данных Wine, чтобы предсказать качество вина на основе его химических свойств. Выберите две модели из пройденных и сравните их эффективность.

Заключение

